

# CustomPromptPage

for IBM Cognos Analytics

Technical Documentation & Configuration Guide

---

Version 9.0

February 2026

**CONFIDENTIAL**

# 1. Introduction

---

The CustomPromptPage is a modular JavaScript framework that replaces native IBM Cognos Analytics prompt pages with a modern, drag-and-drop interface. It allows report authors to define parameter buttons in a JSON configuration, which users then drag from a left navigation pane into interactive cards on a right workspace pane.

This documentation covers the complete system architecture, JSON configuration reference, Cognos report setup requirements, and configuration generation methods.

## 1.1 Key Features

- Drag-and-drop parameter selection with double-click support
- Bubble-based multi-value input with autocomplete from DataStore queries
- Six prompt types: value, text, date, dateRange, dateFromTo, searchSelect
- Search & Select integration for large datasets (2,000+ values)
- Multilingual label support (e.g. EN, DE) with automatic locale detection
- Collapsible groups and subgroups with SVG icons
- Preset configurations for quick one-click parameter loading
- Required field validation with visual indicators
- Paste support for bulk value entry (newline, comma, semicolon, tab delimiters)

## 1.2 Prerequisites

- IBM Cognos Analytics 11.x or later
- **"UI with event propagation"** must be enabled in report properties for drag-and-drop to work across iframe boundaries
- JavaScript module files hosted on an accessible web server or Cognos install location

## 2. Architecture

The system is composed of five JavaScript modules, one CSS file, and a JSON configuration object passed via the Cognos CustomControl configuration property.

### 2.1 Module Overview

| Module              | Role         | Description                                                                                                                                                             |
|---------------------|--------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| preLoad.js          | Bootstrap    | Entry point loaded by Cognos. Dynamically loads CustomPromptPage.js and delegates all lifecycle methods (initialize, draw, getParameters, setData, destroy).            |
| CustomPromptPage.js | Orchestrator | Creates the layout container, loads LeftPane, RightPane, and DragNDrop modules sequentially. Forwards DataStore data and manages the Cognos lifecycle.                  |
| LeftPane.js         | Navigation   | Renders collapsible button groups, subgroups, preset buttons, and localized labels from the JSON configuration. Each button stores its full config for card creation.   |
| RightPane.js        | Workspace    | Renders parameter cards with prompt-type-specific inputs (bubble, date picker, search). Manages parameter collection via getParameters() and required field validation. |
| DragNDrop.js        | Interaction  | Handles mouse-based drag, double-click, preset application, and auto-rendering of required cards. Manages button disable state and floating drag element.               |
| LeftPane.css        | Styling      | Complete stylesheet for left pane, right pane cards, bubbles, date inputs, search & select UI, drop hover states, and responsive layout.                                |

### 2.2 Lifecycle Flow

Cognos calls the control lifecycle methods in this order:

1. `initialize(oControlHost, fnDoneInitializing)` — Modules loaded sequentially: LeftPane → RightPane → DragNDrop
2. `setData(oControlHost, oDataStore)` — Called once per DataStore query; results are cached for autocomplete
3. `draw(oControlHost)` — DataStores forwarded to RightPane; LeftPane and RightPane rendered; DragDrop handlers attached
4. `getParameters(oControlHost)` — Called on Finish/Next; aggregates all card parameters
5. `destroy(oControlHost)` — Cleanup on reprompt (all state is lost; control is recreated)

## 3. JSON Configuration Reference

The entire UI is driven by a single JSON object set in the Cognos CustomControl configuration property. The configuration can be authored manually or generated using the AI prompt template and JSON Schema provided in Section 7.

### 3.1 Root-Level Properties

| Property                 | Type    | Required   | Description                                                                                       |
|--------------------------|---------|------------|---------------------------------------------------------------------------------------------------|
| selectFirstGroupExpanded | Boolean | No         | If true, auto-expands the first button group on load. Overridden by each group's defaultExpanded. |
| presets                  | Array   | No         | Array of preset configurations for one-click parameter loading.                                   |
| buttonGroups             | Array   | <b>Yes</b> | Top-level array of button groups. Each group becomes a collapsible section.                       |

### 3.2 Button Group Properties

| Property        | Type    | Required   | Description                                                                                       |
|-----------------|---------|------------|---------------------------------------------------------------------------------------------------|
| groupLabel      | String  | <b>Yes</b> | Fallback display label for the group header.                                                      |
| groupLabels     | Object  | No         | Localized labels keyed by 2-letter locale (e.g. {"de": "...", "en": "..."})                       |
| groupIcon       | String  | No         | Inline SVG string rendered in the group header.                                                   |
| defaultExpanded | Boolean | No         | Whether this group starts expanded. Default: false.                                               |
| groupItems      | Array   | <b>Yes</b> | Mixed array of subgroups (objects with subgroupLabel + buttons) and/or standalone button objects. |

**Important:** The property name is groupItems (not buttons). Earlier versions used buttons directly on the group; the current version requires groupItems which supports both standalone buttons and subgroups.

### 3.3 Subgroup Properties

| Property        | Type    | Required   | Description                                         |
|-----------------|---------|------------|-----------------------------------------------------|
| subgroupLabel   | String  | <b>Yes</b> | Fallback label for the collapsible subgroup header. |
| subgroupLabels  | Object  | No         | Localized subgroup labels.                          |
| defaultExpanded | Boolean | No         | Whether subgroup starts expanded. Default: false.   |
| buttons         | Array   | <b>Yes</b> | Array of button objects within this subgroup.       |

## 3.4 Prompt Types

Every button should specify a `promptType` to clearly indicate the expected input behavior. The following six types are supported:

| <code>promptType</code>   | Parameter Style                       | Input UI                                 | Use Case                                                                                 |
|---------------------------|---------------------------------------|------------------------------------------|------------------------------------------------------------------------------------------|
| <code>value</code>        | <code>paramName + queryName</code>    | Bubble input with DataStore autocomplete | Standard value prompt with dropdown suggestions from a query                             |
| <code>text</code>         | <code>paramName</code> only           | Bubble input, no autocomplete            | Manual free-text entry when no query is available or dataset is too large                |
| <code>date</code>         | <code>paramName</code> (single)       | Single date picker                       | Single date selection for a parameter                                                    |
| <code>dateRange</code>    | <code>paramName</code> (single)       | FROM/TO date pickers                     | Date range via <code>in_range()</code> — one parameter with start/end structure          |
| <code>dateFromTo</code>   | <code>paramNames.from / .to</code>    | FROM/TO date pickers                     | Date range via <code>BETWEEN</code> — two separate parameters for start and end          |
| <code>searchSelect</code> | <code>paramName + sspBlockName</code> | On-demand search UI                      | Large datasets (2,000+ values); delegates to hidden native Cognos Search & Select prompt |

## 3.5 Button Properties (All Types)

| Property                   | Type    | Req. | Description                                                                                                                                         |
|----------------------------|---------|------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>label</code>         | String  | Yes  | Fallback button display label.                                                                                                                      |
| <code>labels</code>        | Object  | No   | Localized labels: <code>{"de": "Produktkategorie", "en": "Product Category"}</code>                                                                 |
| <code>tooltip</code>       | String  | No   | Fallback hover tooltip text.                                                                                                                        |
| <code>tooltips</code>      | Object  | No   | Localized tooltips by locale code.                                                                                                                  |
| <code>helpText</code>      | String  | No   | Fallback help text shown on the card.                                                                                                               |
| <code>helpTexts</code>     | Object  | No   | Localized help texts.                                                                                                                               |
| <code>paramName</code>     | String  | *    | Cognos parameter name. Required for all types except <code>dateFromTo</code> .                                                                      |
| <code>paramNames</code>    | Object  | *    | Required only for <code>dateFromTo</code> : <code>{"from": "P_DateFrom", "to": "P_DateTo"}</code>                                                   |
| <code>promptType</code>    | String  | Rec. | One of: <code>value</code> , <code>text</code> , <code>date</code> , <code>dateRange</code> , <code>dateFromTo</code> , <code>searchSelect</code> . |
| <code>queryName</code>     | String  | No   | DataStore query name for autocomplete ( <code>value</code> type). Must match a configured DataStore.                                                |
| <code>useColumn</code>     | Integer | No   | Column index for the "use" value sent to Cognos. Default: 0.                                                                                        |
| <code>displayColumn</code> | Integer | No   | Column index for the "display" value shown to user. Default: same as <code>useColumn</code> .                                                       |

|                |         |    |                                                                                                               |
|----------------|---------|----|---------------------------------------------------------------------------------------------------------------|
| required       | Boolean | No | If true, card auto-renders on load and cannot be removed.                                                     |
| sspBlockName   | String  | ** | Required for searchSelect. The "lid" of the hidden Cognos block containing the native Search & Select prompt. |
| useValueLength | Integer | No | For searchSelect: leading characters from result used as "use" value. Omit for full string.                   |
| dateFormat     | String  | No | Informational date format hint (e.g. "yyyy-MM-dd").                                                           |

## 4. Cognos Report Setup

The CustomPromptPage relies on specific Cognos report elements being configured correctly. This section describes the naming conventions and structural requirements.

### 4.1 Report Property: UI with Event Propagation

**Critical:** In the Cognos report properties, the setting **"UI with event propagation"** must be enabled. Without this, drag-and-drop events cannot cross the iframe boundary between the custom control and the report page.

### 4.2 DataStores for Value Prompts and Query Prompts

Buttons with promptType "value" (or any button with a queryName) require a corresponding DataStore to be configured in the Cognos report. The DataStore provides autocomplete suggestions to the user.

#### 4.2.1 Naming Convention

The queryName property in the JSON configuration must exactly match the DataStore name in Cognos. For example, if the button config specifies:

```
"queryName": "qp_ProdCategory"
```

Then the Cognos report must contain a DataStore named "qp\_ProdCategory" with the appropriate query columns dragged into its configuration.

#### 4.2.2 Column Configuration

Each DataStore query must have its required columns explicitly dragged into the DataStore configuration panel. Cognos ignores data items not explicitly included. The useColumn and displayColumn indices in the JSON refer to the zero-based column order in the DataStore.

Example: A DataStore with two columns where column 0 is the code ("PRD01") and column 1 is the description ("PRD01 - Electronics") would use:

```
"useColumn": 0,  
"displayColumn": 1
```

#### 4.2.3 Performance Note

**Important:** DataStore queries auto-execute at page load regardless of whether the user needs that parameter. For datasets exceeding approximately 2,000 values, consider using promptType: "searchSelect" instead, which performs on-demand queries only when the user searches.

### 4.3 Search & Select Prompt Setup

Buttons with promptType "searchSelect" delegate their search functionality to a native Cognos Search & Select prompt that is hidden from the user. This requires specific structural elements in the report.

### 4.3.1 Block with None-Block Display

A Cognos **Block** element must be added to the prompt page with its **Box type** set to "**None-Block**". This hides the block visually while keeping the native prompt functional and accessible to JavaScript.

**Warning:** Do NOT use Box type "None" as this causes Cognos to treat it as a conditional render, and the prompt control will not be included in the page output at all.

### 4.3.2 Block Naming (lid)

The Block element must have its **Name** (which becomes the `lid` attribute in the rendered HTML) set to match the `sspBlockName` property in the JSON configuration. For example:

```
JSON:      "sspBlockName": "b_SSP_CustNR"  
Cognos:    Block Name = "b_SSP_CustNR"
```

Inside this block, place a standard Cognos Search & Select prompt control configured against the desired query subject.

### 4.3.3 useValueLength

Search & Select results typically return composite strings like "CUST001 - Contoso Ltd". The `useValueLength` property extracts only the leading N characters as the "use" value sent to Cognos, while the full string becomes the "display" value. If omitted, the full string is used for both.



## 5. Localization

The system supports multilingual labels at every level: group headers, subgroup headers, button labels, tooltips, and help texts. The locale is detected automatically from `oControlHost.locale` (first two characters).

### 5.1 Localization Pattern

Every text property follows a consistent singular/plural pattern:

| Fallback (singular) | Localized (plural) | Example                                                     |
|---------------------|--------------------|-------------------------------------------------------------|
| label               | labels             | {"de": "Produktkategorie", "en": "Product Category"}        |
| tooltip             | tooltips           | {"de": "Kategorie auswählen", "en": "Select category"}      |
| helpText            | helpTexts          | {"de": "Mindestens einen Wert", "en": "At least one value"} |
| groupLabel          | groupLabels        | {"de": "Produkt (PR)", "en": "Product (PR)"}                |
| subgroupLabel       | subgroupLabels     | {"de": "Produktkategorie", "en": "Product Category"}        |

Fallback priority: locale match > "en" fallback > first available key > singular property.

## 6. Complete Configuration Example

The following example demonstrates a full configuration with two button groups covering all six prompt types. Group 1 (Product) contains a subgroup and standalone date/text buttons. Group 2 (Customer Complaints) contains searchSelect and text buttons.

```
{
  "selectFirstGroupExpanded": true,
  "presets": [
    {
      "id": "preset_basic_product",
      "label": "Basic Product Filter",
      "labels": { "de": "Basis Produktfilter", "en": "Basic Product Filter" },
      "description": "Load common product filter parameters",
      "parameters": [
        { "paramName": "P_ProdCat_L1" },
        { "paramName": "P_Region" }
      ]
    }
  ],
  "buttonGroups": [
    {
      "groupLabel": "Product (PR)",
      "groupLabels": { "de": "Produkt (PR)", "en": "Product (PR)" },
      "groupIcon": "<svg>...</svg>",
      "defaultExpanded": true,
      "groupItems": [
        {
          "subgroupLabel": "Product Category",
          "subgroupLabels": { "de": "Produktkategorie", "en": "Product Category" },
          "defaultExpanded": false,
          "buttons": [
            {
              "label": "Product Category (Level 1)",
              "labels": { "de": "Produktkategorie (Ebene 1)", "en": "Product
Category (Level 1)" },
              "paramName": "P_ProdCat_L1",
              "promptType": "value",
              "queryName": "qp_ProdCat_L1",
              "useColumn": 0,
              "displayColumn": 1,
              "required": true,
              "helpText": "Required: Select at least one category",
              "helpTexts": { "de": "Erforderlich: Mindestens eine Kategorie", "en":
"Required: Select at least one category" }
            }
          ]
        },
        {
          "label": "Order Date (Range)",
          "labels": { "de": "Bestelldatum (Bereich)", "en": "Order Date (Range)" },
          "paramName": "P_OrderDateRange",

```

```
        "promptType": "dateRange",
        "dateFormat": "yyyy-MM-dd"
      },
      {
        "label": "Shipping Date",
        "labels": { "de": "Versanddatum", "en": "Shipping Date" },
        "paramName": "P_ShipDate",
        "promptType": "date"
      }
    ]
  },
  {
    "groupLabel": "Customer Complaints (CC)",
    "groupLabels": { "de": "Kundenbeschwerden (KB)", "en": "Customer Complaints (CC)" },
    "defaultExpanded": false,
    "groupItems": [
      {
        "label": "Customer Number",
        "labels": { "de": "Kundennummer", "en": "Customer Number" },
        "paramName": "P_CustomerNR",
        "promptType": "searchSelect",
        "sspBlockName": "b_SSP_CustNR",
        "useValueLength": 4,
        "required": true
      },
      {
        "label": "Complaint Reference",
        "labels": { "de": "Beschwerdereferenz", "en": "Complaint Reference" },
        "paramName": "P_ComplaintRef",
        "promptType": "text"
      }
    ]
  }
]
```

## 7. AI-Assisted Configuration Generation

In addition to manually authoring the JSON configuration, a JSON Schema and a fill-in prompt template are provided. These allow any AI assistant (Claude, ChatGPT, etc.) to generate a valid configuration from a natural-language description.

### 7.1 Method Overview

| Method             | When to Use                                              | Artifacts Needed                                |
|--------------------|----------------------------------------------------------|-------------------------------------------------|
| <b>Manual JSON</b> | Full control, small configs, iterative tweaks.           | This documentation as reference.                |
| <b>AI + Schema</b> | Large configs, rapid prototyping, non-technical authors. | JSON Schema file + Prompt Template (see below). |

### 7.2 How to Use the AI Prompt

6. Copy the fill-in prompt template (provided as a separate artifact).
7. Replace all [PLACEHOLDER] fields with your actual group names, parameter names, prompt types, and language codes.
8. Paste the completed prompt along with the JSON Schema into any AI assistant.
9. The AI produces a valid JSON configuration ready to paste into the Cognos CustomControl configuration property.
10. Validate the output against the JSON Schema to catch any structural issues.

### 7.3 Prompt Template (Excerpt)

I want to get a CustomPromptPage JSON configuration based on:

LANGUAGES: [en, de]

BUTTON GROUPS:

Group: [Group Name]

Icon: [describe or "none"]

Expanded by default: [yes/no]

Items:

- [Button Label] | param: [P\_Param] | type: [value] | query: [qp\_Name]
- [Button Label] | param: [P\_Param] | type: [searchSelect] | ...

PRESETS (optional):

- [Preset Label]: [P\_Param1, P\_Param2]

Generate valid JSON following the CustomPromptPage JSON Schema.

The full prompt template and JSON Schema are provided as separate deliverables alongside this documentation.

## 8. Known Constraints & Considerations

### 8.1 CustomControl Lifecycle

Cognos destroys and recreates the CustomControl on every reprompt cycle (destroy → initialize → draw). All JavaScript state, including card selections and bubble values, is lost. This is an inherent limitation of the CustomControl API. The PageModule architecture (which persists via hide/show) is being explored as a future alternative.

### 8.2 DataStore Auto-Execution

All DataStore queries execute at page load, regardless of whether the user actually needs that parameter. For reports with many DataStores or large datasets, this causes noticeable load time. Mitigation: use "searchSelect" for large datasets, and only configure DataStores for parameters that genuinely benefit from autocomplete.

### 8.3 Unicode in Configuration Files

When editing JSON configuration files containing special characters (German umlauts, Unicode symbols like ×, ✓, ☆), ensure the file is saved with UTF-8 encoding. Cognos may not preserve encoding correctly if the file passes through non-UTF-8 aware tooling.

### 8.4 iframe Boundaries

The CustomControl renders inside an iframe. Mouse events during drag operations must propagate across this boundary. This is why the **"UI with event propagation"** report property is mandatory. Without it, drag events are silently dropped at the iframe border.

## 9. Glossary

| Term                      | Definition                                                                                                                                                             |
|---------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>DataStore</b>          | A Cognos report element that executes a query and makes results available to JavaScript via <code>oControlHost.getDataStore()</code> .                                 |
| <b>Block (None-Block)</b> | A Cognos layout container with Box type "None-Block" that is invisible but still rendered in the DOM, allowing JavaScript to access its child elements.                |
| <b>lid</b>                | The "Name" property of a Cognos element, rendered as the lid attribute in HTML. Used to locate elements via <code>document.querySelector('[lid="..."]')</code> .       |
| <b>oControlHost</b>       | The Cognos-provided interface object passed to every lifecycle method. Provides access to configuration, container, DataStores, locale, and parameter methods.         |
| <b>Bubble</b>             | A visual chip/tag UI element representing a confirmed parameter value. Created by typing + Enter, selecting from autocomplete, or pasting. Removable via the x button. |
| <b>valueChanged()</b>     | Method on <code>oControlHost</code> that notifies Cognos the control's parameter values have changed, triggering internal state updates.                               |
| <b>Preset</b>             | A pre-defined combination of parameters that can be loaded with a single                                                                                               |

|  |                                                       |
|--|-------------------------------------------------------|
|  | click, auto-creating cards for each listed paramName. |
|--|-------------------------------------------------------|